Stateless reports via Micrometer → Prometheus/Grafana (optional)

A metrics-native alternative to three of the seven Flink reports: emit them as Micrometer meters from the producer interceptor and let Prometheus do the windowing at read time. **Additive and opt-in** — it does not replace the Flink reports. The companion one-command showcase (Prometheus + Grafana on Minikube) has its own runbook: k8s/monitoring/README.md.

Contents

Why three of seven

Is Flink overkill for these reports? **Yes and no** — it's not all about stateless aggregation. Of the seven reports, **three are pure stateless scalar aggregation** keyed on bounded-cardinality dimensions (service / topic / hop_count — never `trace_id`):

- **`latency_1m`** (10_latency_report.fql) — avg/max latency per `(pipeline, origin_service, this_topic)`.
- **`topology_1m`** (20_topology_report.fql) — produce-edge record counts.
- **`hop_distribution_1m`** (30_hop_distribution.fql) — records per `hop_count`.

These don't need a stream processor. The producer interceptor already has every value in scope on each `send()`, so it can emit them as **Micrometer** meters and let **Prometheus** do the 1-minute windowing at query time (`rate()` / `increase()`), with **Grafana** on top. The other four reports — `latency_percentiles` (mergeable T-Digest), `coverage`, `bipartite_topology`, `stuck_trace` — are per-`trace_id` *stateful* or *absence-of-event* problems Prometheus can't express, so they **stay in Flink**. This path is **additive and opt-in** (off by default); it doesn't replace the Flink reports — it's the cheaper way to serve the three that are metrics, not stream processing.

Emission lives in one place — IsotopeMetrics.java — mirroring how `TDigests` centralizes the sketch contract.

Enabling the exporter

Pass `-Dmetrics.prometheus.enabled=true` to any long-running stage. **Producing** stages (`hop` / `enrich` / `fulfill`) emit the produce-side meters via the interceptor; **consuming** stages emit the consume-side meters via the marker path (Consume-side meters) — `hop` does both (it consumes *and* produces), and the terminal `consume` / `ship` stages emit the consume side alone. The app serves the meters at `GET /metrics` on `metrics.prometheus.port` (default `9404`) via the JDK's built-in HTTP server — no extra runtime.

```
# Prereq: 'make kafka-pf-up' is up. Run the enrich stage with metrics
exposed:
./gradlew :app:run -Dmetrics.prometheus.enabled=true --args="enrich"
# → metrics on http://localhost:9404/metrics

# In another shell, drive traffic so the meters move (origin produce):
for i in {1..30}; do ./gradlew :app:run --args="place burst-$i" -q; sleep 5;
done

# Scrape:
curl -s localhost:9404/metrics | grep isotope_
```

Point a Prometheus scrape job at each stage's `:9404` and the three reports become PromQL queries (no report topics, no Control Center — Grafana instead). Unlike the Flink jobs, there's **no watermark wait**: cumulative counters update on every send and Prometheus windows them at read time.

> **Reading the latency metric — mind the backlog.** `hop` / `consume` use a *random* consumer group with `auto.offset.reset=earliest`, so a fresh stage **replays the whole topic from offset 0**. Latency is `now − origin_ts`, so days-old replayed records report enormous values — e.g. an `isotope_hop_latency_seconds_sum` of ~9,000,000 over 58 records (~43 h average) is just the backlog, not steady-state latency (the Flink `latency_1m` report shows the same, by the same definition). The tell: `_count` exceeds the number of records you just sent. Two ways to see realistic latency: drain the backlog and read `rate(isotope_hop_latency_seconds_sum[1m]) / rate(..._count[1m])` (the windowed rate ignores the stale backlog once it stops growing), or skip the backlog entirely with `-Disotope.consume.from=latest` so the stage only consumes records produced after it starts:
>
> ```
> ./gradlew :app:run -Dmetrics.prometheus.enabled=true -
> Disotope.consume.from=latest --args="enrich"
> ```

The three reports as PromQL

Two meters cover all three reports — a `Timer` whose count doubles as the topology edge count, plus a `Counter` for the hop histogram:

Report	Meter (Prometheus name)	Labels
`latency_1m` + `topology_1m`	`isotope_hop_latency_seconds_{count,sum,max}` (`Timer`)	`pipeline`, `origin_service`, `this_service`, `this_topic`
`hop_distribution_1m`	`isotope_hop_records_total` (`Counter`)	`pipeline`, `this_topic`, `hop_count`

```
# latency_1m — avg latency (seconds)
  sum by (pipeline, origin_service, this_topic)
(rate(isotope_hop_latency_seconds_sum[1m]))
/ sum by (pipeline, origin_service, this_topic)
(rate(isotope_hop_latency_seconds_count[1m]))

# latency_1m — max
max by (pipeline, origin_service, this_topic)
(isotope_hop_latency_seconds_max)

# topology_1m — records per produce edge
sum by (pipeline, origin_service, this_service, this_topic)
(increase(isotope_hop_latency_seconds_count[1m]))

# hop_distribution_1m — records per hop_count
sum by (pipeline, this_topic, hop_count)
(increase(isotope_hop_records_total[1m]))
```

Consume-side meters

The three meters above all come from the **produce** side — the interceptor on `send()`. The **consume** side adds three more. Two — the edge counter and the time-to-consume latency — are emitted by `IsotopeContext.recordConsume` right beside the consume-edge marker it writes to `isotope_consume_edge_markers`. The third, `isotope.consume.age`, is emitted **once per consumed record on whichever path the consumer takes**: continuing consumers report it from the **adoption path** (`IsotopeContext.adoptFromRecord(record, service)`), and terminal consumers — which never adopt — report it from the **marker path** (`recordConsume`, guarded on `current() == null` so a stage that does both, like `hop`, never double-counts). So age fires on every consuming stage: `hop` via adoption, the terminal `consume` / `ship` stages via the marker. Same gating — no-op unless the exporter is started.

Signal	Meter (Prometheus name)	Labels
consume-edge counts (topic→consumer)	`isotope_consume_records_total` (`Counter`)	`pipeline, this_topic, consumer_service`
time-to-consume latency (origin→consume)	`isotope_consume_latency_seconds_{count,sum,max}` (`Timer`)	`pipeline, origin_service, consumer_service, this_topic`
origin→adoption age (how stale at pickup)	`isotope_consume_age_seconds_{count,sum,max}` (`Timer`)	`pipeline, origin_service, consumer_service, this_topic`

```
# consume edges — records consumed per (topic → consumer) per minute
sum by (pipeline, this_topic, consumer_service)
(increase(isotope_consume_records_total[1m]))
```

```
# time-to-consume — avg seconds from trace origin to consumption
  sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_latency_seconds_sum[1m]))
/ sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_latency_seconds_count[1m]))

# consume age — avg seconds a record had aged by the time a service adopted
it
  sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_age_seconds_sum[1m]))
/ sum by (pipeline, consumer_service, this_topic)
(rate(isotope_consume_age_seconds_count[1m]))
```

These are **net-new signals**, not a port of a Flink report. `isotope_consume_records_total` is the *consume half* of the bipartite topology — the topic→consumer edge tallies — and `isotope_consume_latency_seconds_*` is an origin→consume figure no Flink report computes (the Flink `latency_1m` measures origin→**produce**-hop). The **full** `bipartite_topology` report still stays in Flink: stitching produce and consume edges per `trace_id` into one graph is per-trace stateful, which a counter can't do. What you get here is the bounded-cardinality edge *counts* and a time-to-consume distribution — cheap, windowable at read time, and free of the watermark wait.

`isotope_consume_age_seconds_*` measures the **same `now − origin_ts` quantity** as `isotope_consume_latency_seconds_*`, but it's the **universal consume-side age signal** — emitted once on *every* consuming stage, where latency only fires on the marker path. That's the point of having both: query `isotope_consume_age_*` to get consume-lag across **all** consumers (including `hop`-style stages that adopt-and-forward without ever writing a marker), and the timer's `_count` doubles as the per-`(consumer_service, topic)` consume rate. At a terminal consumer the age equals that consumer's latency by definition (same `now − origin_ts`); the two diverge in *coverage*, not value — latency is marker-only, age is everywhere.

> The same **backlog caveat** from Enabling the exporter applies: `hop` / `consume` / `ship` use a random consumer group with `auto.offset.reset=earliest`, so a fresh stage replays from offset 0 and both `isotope_consume_latency_seconds_sum` and `isotope_consume_age_seconds_sum` then reflect days-old origins, not steady-state. Read the windowed `rate(...sum[1m]) / rate(...count[1m])`, or start with `-Disotope.consume.from=latest`.

What stays in Flink — and two deliberate gaps

Moving these out isn't free — two columns the Flink reports carry have **no Prometheus equivalent**, and both are documented in IsotopeMetrics.java:

- **No `distinct_traces`.** All three SQL reports carry a `COUNT(DISTINCT trace_id)` column. A counter can't dedup, and `trace_id` is unbounded-cardinality so it can't be a label. If you need it, that column stays in Flink (or approximate it with a HyperLogLog sketch).
- **No windowed `min` latency.** A Micrometer `Timer` exposes max but not a per-window min — avg and max port cleanly, min does not.

So the line between "metric" and "Flink job" runs *through* a couple of these reports, not cleanly between them — which is exactly why the move is opt-in rather than a wholesale replacement.

Why `latency_percentiles_1m` is NOT in this list. Percentiles *can* be served by Prometheus — but only via a *classic* histogram (`publishPercentileHistogram()` + `histogram_quantile()`), whose accuracy is **bucket-bound, not adaptive**: error is the width of fixed, pre-chosen buckets, so the tail (p99) is only as good as your bucket layout, and covering a range finely means emitting hundreds of `le` series per tag combo. Prometheus's adaptive answer — *native histograms* (exponential buckets, the closest thing to a T-Digest) — isn't emittable through Micrometer yet (experimental, protobuf-only as of late 2024). So the T-Digest PTF wins on tail accuracy **and** scales better: its sketch is bounded (~few KB/key) and mergeable, whereas at production volume a Micrometer percentile-histogram's `le`-bucket cardinality grows with the range you need to resolve. The built-in Flink `PERCENTILE` aggregate (exact, pure SQL) is *also* the wrong call at high volume — it retains every value in the window — so percentiles stay a **T-Digest sketch PTF** on purpose (see that file's header for the full rationale). This is a 3-Micrometer / 4-Flink split, not 4/3.

One-command showcase: Prometheus + Grafana on Minikube

To *see* these meters instead of `curl`-ing `/metrics`, there's an optional, self-contained stack under k8s/monitoring/ — Prometheus + Grafana as Minikube pods, with the datasource and a dashboard (all six produce/consume signals above) **auto-provisioned**, so it opens straight to a populated board with no login.

The pipeline stages run on your **host** via `./gradlew :app:run`, not in-cluster — so Prometheus scrapes back across the Minikube→host bridge `host.minikube.internal`, one host port per stage (`enrich`→9410, `fulfill`→9411, `ship`→9412; edit k8s/monitoring/10-prometheus.yaml to change the mapping).

```
make metrics-up        # deploy, wait, port-forward Prometheus+Grafana, open
Grafana

# In separate terminals, run each stage on its mapped port (metrics on):
./gradlew :app:run -Dmetrics.prometheus.enabled=true -
Dmetrics.prometheus.port=9410 \
  -Disotope.consume.from=latest --args="enrich"     # fulfill→9411, ship→9412
# …then drive traffic with `place` so the meters move.

make metrics-down      # stop the port-forwards (pods stay up)
make metrics-delete    # tear the whole showcase down
```

Confirm scraping at Prometheus → Status → Targets (a target is `DOWN` until you start its stage — expected). Full runbook and troubleshooting: k8s/monitoring/README.md.